

TECHNICAL DOCUMENTATION

RL Fighting Project

Reinforcement Learning Combat Simulation with Q-Learning and Double Q-Learning

Version: 1.0

Date: May 2026

Modules: 6 Python Modules

Table of Contents

1. Introduction

- 1.1 Project Overview
- 1.2 Objectives and Scope
- 1.3 Technology Stack

2. System Architecture

- 2.1 Component Overview
- 2.2 Data Flow

3. Environment Module

- 3.1 FightingEnv Class
- 3.2 State Space
- 3.3 Action Space
- 3.4 Reward Function

4. Agents Module

- 4.1 QLearningAgent
- 4.2 DoubleQLearningAgent
- 4.3 Algorithm Comparison

5. Training Module

- 5.1 Single Agent Training
- 5.2 Stability Analysis

6. Visualization Module

- 6.1 Combined Analysis Plot
- 6.2 Individual Plot Functions

7. Animation Module

- 7.1 PyGame Visualization
- 7.2 Video Export

8. Main Execution Flow

9. Hyperparameters and Configuration

10. Usage Guide

10.1 Installation

10.2 Running the Project

10.3 Customization

Appendix A: Mathematical Foundations

Appendix B: Quick Reference

1. Introduction

1.1 Project Overview

The **RL Fighting Project** is a comprehensive reinforcement learning framework that trains intelligent combat agents using Q-Learning and Double Q-Learning algorithms. The project simulates a turn-based fighting environment where an AI agent learns optimal combat strategies against a randomized opponent through episodic training.

The system is implemented across six modular Python components, each responsible for a distinct aspect of the reinforcement learning pipeline: environment simulation, agent decision-making, training orchestration, data visualization, and animated rendering. This modular architecture ensures maintainability, extensibility, and clear separation of concerns.

1.2 Objectives and Scope

The primary objectives of this project are:

- **Algorithm Implementation:** Implement and compare standard Q-Learning against Double Q-Learning in a combat scenario.
- **Environment Design:** Create a turn-based fighting simulation with meaningful state representation, action choices, and reward shaping.
- **Training Pipeline:** Develop a robust training loop with progress monitoring and stability analysis across multiple independent trials.
- **Analysis and Visualization:** Provide comprehensive plotting tools for learning curves, Q-value heatmaps, policy visualization, and algorithm comparison.
- **Demonstration:** Include animated fight visualization with video export capability for result presentation.

1.3 Technology Stack

Table 1 Technology Stack and Dependencies

Component	Library	Purpose
Core Computation	NumPy	Numerical operations, Q-table storage
Standard Library	random	Exploration and opponent randomization
Visualization	Matplotlib	Learning curves, policy plots, comparisons
Visualization	Seaborn	Q-value heatmap rendering
Animation	PyGame	Real-time fight visualization
Video Export	imageio	MP4 video generation from frames

2. System Architecture

2.1 Component Overview

The project follows a layered architecture with six core modules. Each module encapsulates a specific responsibility within the reinforcement learning pipeline:

Table 2 Module Responsibilities

Module	File	Primary Responsibility
Environment	<code>environment.py</code>	Combat simulation, state transitions, reward computation
Agents	<code>agents.py</code>	Q-Learning and Double Q-Learning agent implementations
Training	<code>training.py</code>	Training loops and multi-trial stability analysis
Visualization	<code>visualization.py</code>	Matplotlib-based analysis plots
Animation	<code>animation.py</code>	PyGame fight rendering and video export
Main	<code>main.py</code>	Entry point, orchestrates full pipeline

2.2 Data Flow

The execution flow follows a standard reinforcement learning pattern. The **Main** module instantiates the **Environment** and **Agent**, then invokes the **Training** module to execute the learning loop. During each episode, the agent observes the current state, selects an action (using epsilon-greedy exploration), receives the next state and reward from the environment, and updates its Q-values accordingly. After training completes, the **Visualization** and **Animation** modules consume the trained agent and training history to produce analytical outputs.

3. Environment Module

The `environment.py` module defines the `FightingEnv` class, which implements the combat simulation as a Markov Decision Process (MDP). The environment manages health points, action resolution, blocking mechanics, and reward computation.

3.1 FightingEnv Class

Class: `FightingEnv`

Simulates a turn-based combat scenario between an AI agent and a randomized opponent. Both combatants start with 1000 HP and take turns executing actions until one is defeated.

Constructor Parameters

Table 3 `FightingEnv` Constructor Parameters

Parameter	Type	Default	Description
<code>block_prob</code>	float	0.2	Probability that the opponent blocks an attack

Key Attributes

Table 4 FightingEnv Internal Attributes

Attribute	Type	Description
<code>max_hp</code>	int	Maximum health points (1000)
<code>agent_hp</code>	int	Current agent health
<code>opponent_hp</code>	int	Current opponent health
<code>damage_map</code>	dict	Action-to-damage mapping: {0: 10, 1: 15, 2: 25}
<code>block_prob</code>	float	Opponent block probability
<code>steps</code>	int	Episode step counter

Core Methods

reset() - Resets the environment to initial state. Returns the initial state tuple `(agent_hp // 10, opponent_hp // 10)`.

step(agent_action) - Executes one turn of combat:

1. Agent executes chosen action; damage is applied if not blocked.
2. Opponent randomly selects and executes a counter-action.
3. Health points are updated; terminal conditions are checked.
4. Returns: `(next_state, reward, done, info_dict)`.

3.2 State Space

The state representation is a discretized tuple `(agent_hp_scaled, opponent_hp_scaled)` where each health value is divided by 10 and truncated to an integer. This maps the continuous health range of 0-1000 into a discrete grid of 101 x 101 possible states, making tabular Q-learning computationally feasible.

$$s = (\lfloor \text{HP}_{\text{agent}} / 10 \rfloor, \lfloor \text{HP}_{\text{opponent}} / 10 \rfloor), \quad s \in \{0, \dots, 100\} \times \{0, \dots, 100\} \quad (1)$$

3.3 Action Space

The environment supports three discrete actions, each corresponding to a different attack type with distinct damage values:

Table 5 Action Space Definition

Action ID	Name	Damage	Description
0	HAND	10	Light attack, low damage
1	FOOT	15	Medium attack, moderate damage
2	WEAPON	25	Heavy attack, high damage

3.4 Reward Function

The reward function is carefully shaped to encourage aggressive yet strategic behavior. The reward at each step is computed as:

$$r = r_{\text{damage}} + r_{\text{blocked}} + r_{\text{incoming}} + r_{\text{terminal}} + r_{\text{step}} \quad (2)$$

Table 6 Reward Component Breakdown

Component	Condition	Value	Purpose
<code>r_damage</code>	Attack lands	$+\text{damage} * 5$	Reward for dealing damage
<code>r_blocked</code>	Attack blocked	-20	Penalty for failed attack
<code>r_incoming</code>	Always	$-\text{opp_damage} * 3$	Penalty for taking damage
<code>r_terminal</code>	Agent wins	+1000	Large win bonus
<code>r_terminal</code>	Agent loses	-1000	Large loss penalty
<code>r_step</code>	Always	-1	Small time penalty

4. Agents Module

The `agents.py` module implements two reinforcement learning algorithms: standard Q-Learning and Double Q-Learning. Both agents use tabular value function approximation with epsilon-greedy exploration.

4.1 QLearningAgent

Class: QLearningAgent

Implements standard one-step Q-Learning with epsilon-greedy action selection and exponential epsilon decay. Maintains a single Q-table of shape (101, 101, 3).

Constructor Parameters

Table 7 QLearningAgent Constructor Parameters

Parameter	Type	Default	Description
<code>state_dim</code>	tuple	(101, 101)	Discretized state space dimensions
<code>n_actions</code>	int	3	Number of available actions
<code>alpha</code>	float	0.1	Learning rate
<code>gamma</code>	float	0.95	Discount factor
<code>epsilon</code>	float	0.2	Initial exploration rate
<code>epsilon_min</code>	float	0.01	Minimum exploration rate
<code>epsilon_decay</code>	float	0.995	Exploration decay per update

Core Methods

`act(state, eval_mode=False)` - Selects an action using epsilon-greedy policy. If `eval_mode=True`, always selects the greedy action (no exploration).

`update(state, action, reward, next_state, done)` - Performs a single Q-Learning update using the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (3)$$

After each update, epsilon is decayed multiplicatively: `epsilon *= epsilon_decay`, clamped at `epsilon_min`.

4.2 DoubleQLearningAgent

Class: DoubleQLearningAgent

Implements Double Q-Learning, which maintains two separate Q-tables (Q1 and Q2) to mitigate overestimation bias. On each update, one table is selected randomly for update while the other provides the target value.

Constructor Parameters

Identical to `QLearningAgent` (see Table 7).

Core Methods

`act(state, eval_mode=False)` - Selects action based on the averaged Q-values: `argmax((Q1 + Q2) / 2)`.

`update(state, action, reward, next_state, done)` - With probability 0.5, updates Q1 using Q2 for target value; otherwise updates Q2 using Q1. The update rule for Q1 is:

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha [r + \gamma Q_2(s', \arg\max_{a'} Q_1(s', a')) - Q_1(s, a)] \quad (4)$$

The Q2 update is symmetric. This decoupling of action selection from value estimation reduces the overestimation bias inherent in standard Q-Learning.

4.3 Algorithm Comparison

Table 8 Q-Learning vs Double Q-Learning Comparison

Aspect	Q-Learning	Double Q-Learning
Q-Tables	Single table	Two tables (Q1, Q2)
Target computation	max over same table	max over one table, value from other
Overestimation bias	Present	Reduced
Action selection	argmax of Q-table	argmax of averaged tables
Memory usage	101 x 101 x 3 floats	2 x (101 x 101 x 3) floats
Update complexity	$O(A)$	$O(A)$

5. Training Module

The `training.py` module provides the core training infrastructure, including single-agent training loops and multi-trial stability analysis.

5.1 Single Agent Training

Function: `train_agent(env, agent, episodes, verbose)`

Trains a single agent on the provided environment for a specified number of episodes using the standard episodic reinforcement learning loop.

Parameters

Table 9 train_agent Parameters

Parameter	Type	Default	Description
env	FightingEnv	required	Environment instance
agent	Agent	required	QLearningAgent or DoubleQLearningAgent
episodes	int	500	Number of training episodes
verbose	bool	True	Print progress every 100 episodes

Training Loop Algorithm

Algorithm: Episodic Q-Learning Training

```

Initialize rewards list
For each episode from 1 to episodes:
    state ← env.reset()
    total_reward ← 0
    done ← False
    While not done:
        action ← agent.act(state)
        next_state, reward, done, _ ← env.step(action)
        agent.update(state, action, reward, next_state, done)
        state ← next_state
        total_reward ← total_reward + reward
    Append total_reward to rewards
Return rewards

```

5.2 Stability Analysis

Function: `run_multiple_trials(env_class, agent_class, n_trials, episodes, agent_kwargs)`

Runs multiple independent training trials to assess algorithm stability and variance. Returns a 2D array of shape (n_trials, episodes) containing per-episode rewards for statistical analysis.

Parameters

Table 10 `run_multiple_trials` Parameters

Parameter	Type	Default	Description
<code>env_class</code>	class	required	Environment class (not instance)
<code>agent_class</code>	class	required	Agent class (not instance)
<code>n_trials</code>	int	3	Number of independent trials
<code>episodes</code>	int	200	Episodes per trial
<code>agent_kwargs</code>	dict	{}	Keyword arguments for agent constructor

The function pads shorter episodes (due to early termination) by repeating the final reward value, ensuring consistent array dimensions for statistical aggregation.

6. Visualization Module

The `visualization.py` module provides comprehensive plotting capabilities using Matplotlib and Seaborn. It includes a unified dashboard function as well as individual plotting utilities.

6.1 Combined Analysis Plot

Function: `plot_all_in_one(...)`

Generates a comprehensive 2x3 figure containing all analysis visualizations in a single view. This is the primary analysis tool for comparing algorithm performance.

Subplot Layout

Table 11 `plot_all_in_one` Subplot Layout

Position	Plot Type	Content
(0, 0)	Learning Curves	Smoothed rewards for both algorithms
(0, 1)	Q-Value Heatmap	Seaborn heatmap of Q-values at fixed agent HP
(0, 2)	Policy Visualization	Argmax policy grid over state space
(1, 0)	Stability Plot	Q-Learning mean and std across trials
(1, 1)	Stability Plot	Double Q-Learning mean and std across trials
(1, 2)	Final Comparison	Raw and smoothed side-by-side comparison

6.2 Individual Plot Functions

The module also exports standalone functions for flexible analysis:

Table 12 Individual Plot Functions

Function	Purpose
<code>plot_learning_curve(rewards, title)</code>	Single algorithm learning curve with moving average
<code>plot_q_heatmap(q_table, agent_hp, title)</code>	Q-value heatmap for a specific agent HP level
<code>plot_policy(policy_grid, title)</code>	2D policy visualization with color-coded actions
<code>plot_stability(all_rewards, algorithm_name)</code>	Mean and standard deviation across trials
<code>compare_algorithms(results_dict)</code>	Multi-algorithm comparison plot

7. Animation Module

The `animation.py` module provides a PyGame-based real-time visualization of trained agent combat, with optional video export via imageio.

7.1 PyGame Visualization

Function: `show_fight(env, agent, delay, save_video, video_filename)`

Renders a real-time animated fight between the trained agent (blue) and a random opponent (red). Displays health bars, action labels, damage numbers, and block notifications.

Parameters

Table 13 `show_fight` Parameters

Parameter	Type	Default	Description
<code>env</code>	FightingEnv	required	Environment instance
<code>agent</code>	Agent	required	Trained agent (eval_mode used)
<code>delay</code>	float	1.0	Seconds per frame
<code>save_video</code>	bool	True	Export to MP4
<code>video_filename</code>	str	"fight_video.mp4"	Output video filename

Visual Elements

The renderer draws human-like stick figures with the following visual feedback:

- **Health Bars:** Green bars above each character, shrinking with damage.
- **Action Labels:** Text indicating the current action (HAND, FOOT, WEAPON).
- **Damage Numbers:** Floating red numbers showing damage dealt.
- **Block Indicator:** Warning text when an attack is blocked.
- **Result Banner:** Win/loss announcement at fight conclusion.

7.2 Video Export

When `save_video=True`, each rendered frame is captured as a NumPy array and written to an MP4 file using `imageio`. The frame rate is determined by `fps = 1 / delay`. The video is saved after the fight concludes with a 2-second game-over screen.

8. Main Execution Flow

The `main.py` module serves as the entry point, orchestrating the complete pipeline from training through analysis. The execution proceeds through six sequential stages:

Table 14 Main Execution Pipeline

Stage	Description	Output
1. Q-Learning Training	Train QLearningAgent for 1000 episodes	q_rewards list
2. Double Q-Learning Training	Train DoubleQLearningAgent for 1000 episodes	dq_rewards list
3. Stability Trials	Run 5 trials of 300 episodes each for both algorithms	q_stability, dq_stability arrays
4. Combined Visualization	Generate 6-panel analysis figure	Matplotlib figure
5. Fight Animation	Render trained agent combat with video export	fight_video.mp4
6. Refinement Log	Display example refinement notes	Console output

Default Hyperparameters

```

EPISODES      = 1000    # Training episodes per algorithm
TRIALS        = 5       # Independent stability trials
TRIAL_EPISODES = 300    # Episodes per stability trial

Agent Parameters:
  alpha        = 0.1     # Learning rate
  gamma        = 0.95    # Discount factor
  epsilon      = 0.2     # Initial exploration rate
  epsilon_min  = 0.01    # Minimum exploration rate
  epsilon_decay = 0.995  # Exploration decay factor

```

9. Hyperparameters and Configuration

Environment Hyperparameters

Table 15 Environment Configuration

Parameter	Default	Range	Effect
max_hp	1000	100 - 5000	Episode length scaling
block_prob	0.2	0.0 - 1.0	Opponent defense strength
damage_map[0]	10	1 - 100	Light attack damage
damage_map[1]	15	1 - 100	Medium attack damage
damage_map[2]	25	1 - 100	Heavy attack damage

Agent Hyperparameters

Table 16 Agent Configuration

Parameter	Default	Range	Effect
alpha	0.1	0.001 - 1.0	Higher = faster learning, less stable
gamma	0.95	0.0 - 1.0	Higher = more future-oriented
epsilon	0.2	0.0 - 1.0	Initial exploration probability
epsilon_min	0.01	0.0 - 1.0	Floor for exploration rate
epsilon_decay	0.995	0.9 - 1.0	Lower = faster exploitation transition

Training Hyperparameters

Table 17 Training Configuration

Parameter	Default	Description
episodes	1000	Total training episodes
n_trials	5	Stability analysis trials
trial_episodes	300	Episodes per stability trial

10. Usage Guide

10.1 Installation

Install the required dependencies using pip:

```
pip install numpy matplotlib seaborn pygame imageio
```

10.2 Running the Project

Execute the main pipeline:

```
python main.py
```

This will train both agents, run stability trials, generate the analysis figure, and render the fight animation with video export.

10.3 Customization

To modify hyperparameters, edit the values in `main.py`:

```
# Example: Increase learning rate and exploration
q_agent = QLearningAgent(alpha=0.15, gamma=0.99, epsilon=0.3,
                          epsilon_min=0.01, epsilon_decay=0.99)
```

To use individual modules:

```
from environment import FightingEnv
from agents import QLearningAgent
from training import train_agent

env = FightingEnv(block_prob=0.3) # Harder blocking
agent = QLearningAgent(alpha=0.2, gamma=0.9)
rewards = train_agent(env, agent, episodes=500, verbose=True)
```

Appendix A: Mathematical Foundations

A.1 Q-Learning

Q-Learning is a model-free, off-policy temporal difference control algorithm. It iteratively estimates the optimal action-value function $Q^*(s, a)$ by applying the Bellman optimality operator. The update rule at each step is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (\text{A.1})$$

where α is the learning rate and γ is the discount factor. Under standard conditions (sufficient exploration, decaying learning rate), Q-Learning converges to the optimal Q-function with probability 1.

A.2 Double Q-Learning

Double Q-Learning addresses the overestimation bias in standard Q-Learning by decoupling action selection from value estimation. Two Q-tables, Q^A and Q^B , are maintained and updated alternately:

$$Q^A(s_t, a_t) \leftarrow Q^A(s_t, a_t) + \alpha [r_{t+1} + \gamma Q^B(s_{t+1}, \operatorname{argmax}_a Q^A(s_{t+1}, a)) - Q^A(s_t, a_t)] \quad (\text{A.2})$$

This prevents the maximization bias because the action selected by Q^A is evaluated by Q^B , which provides an unbiased estimate.

A.3 Epsilon-Greedy Exploration

Action selection follows the epsilon-greedy policy:

$$\begin{aligned} \pi(a|s) &= 1 - \epsilon + \epsilon/|A| \quad \text{if } a = \operatorname{argmax}_{a'} Q(s, a') \\ \pi(a|s) &= \epsilon/|A| \quad \text{otherwise} \end{aligned} \quad (\text{A.3})$$

With exponential decay: $\epsilon_{t+1} = \max(\epsilon_{\min}, \epsilon_t * \epsilon_{\text{decay}})$.

Appendix B: Quick Reference

B.1 File Structure

```
rl-fighting-project/
|-- main.py           # Entry point and pipeline orchestration
|-- environment.py    # FightingEnv combat simulation
|-- agents.py         # QLearningAgent, DoubleQLearningAgent
|-- training.py       # train_agent, run_multiple_trials
|-- visualization.py  # plot_all_in_one and individual plots
|-- animation.py      # show_fight with PyGame and video export
```

B.2 Class and Function Reference

Table 18 Complete API Reference

Module	Name	Type	Description
environment	<code>FightingEnv</code>	Class	Combat environment
environment	<code>reset()</code>	Method	Reset to initial state
environment	<code>step(action)</code>	Method	Execute one turn
agents	<code>QLearningAgent</code>	Class	Standard Q-Learning
agents	<code>DoubleQLearningAgent</code>	Class	Double Q-Learning
agents	<code>act(state, eval_mode)</code>	Method	Epsilon-greedy action
agents	<code>update(...)</code>	Method	TD learning update
training	<code>train_agent(...)</code>	Function	Single agent training
training	<code>run_multiple_trials(...)</code>	Function	Stability analysis
visualization	<code>plot_all_in_one(...)</code>	Function	Combined dashboard
animation	<code>show_fight(...)</code>	Function	Fight animation

B.3 Reward Formula Quick Reference

```

if attack not blocked:
    reward += damage * 5
else:
    reward -= 20

reward -= opponent_damage * 3 # always applied

if agent_hp <= 0:    reward -= 1000 # loss
if opponent_hp <= 0: reward += 1000 # win

reward -= 1 # per-step time penalty

```